MLOps – HANDS-ON PRACTICAL

Implementing Continuous Integration (CI) with GitHub Actions

Student Practical Documentation

Course	MLOps
Topic	CI Workflow / GitHub Actions
Practical	Automated Python Testing using GitHub Actions
Platform	Ubuntu / WSL2 + Git + GitHub

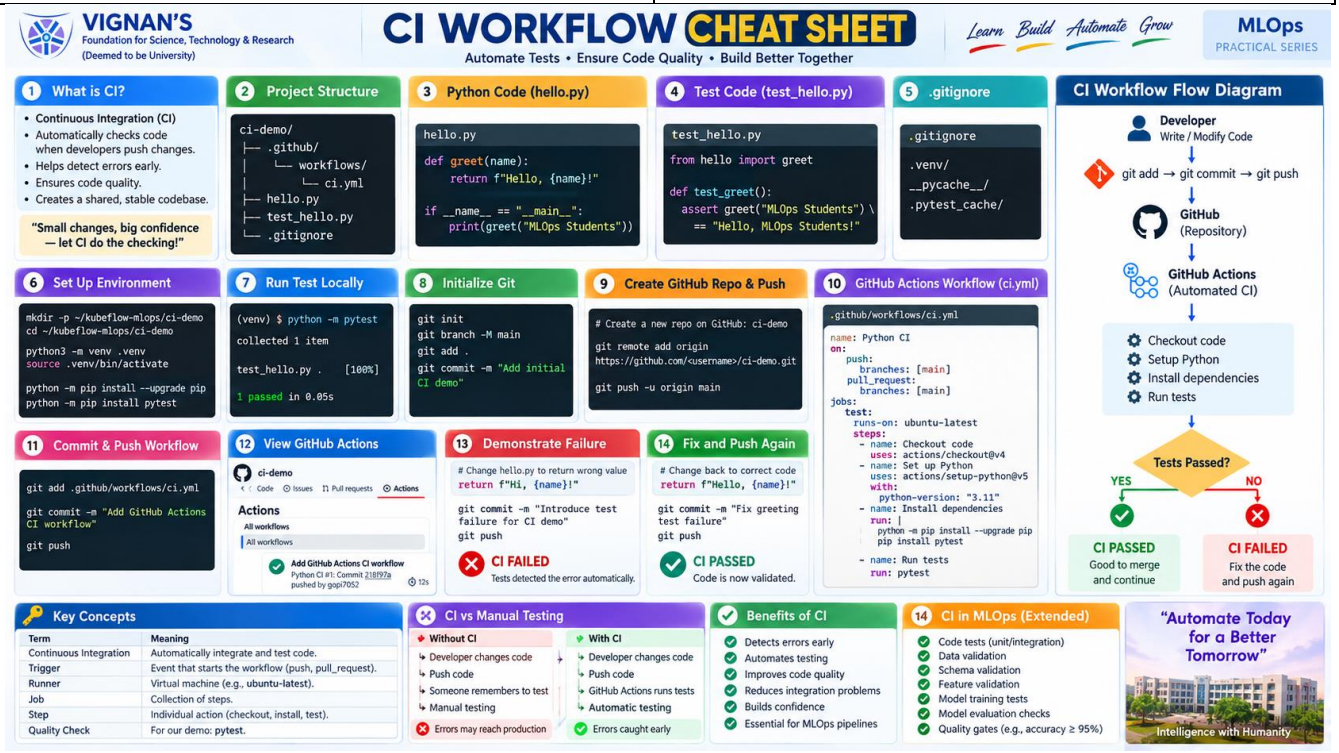


Figure 1. CI Workflow Cheat Sheet

1. Learning Objectives

- Understand Continuous Integration (CI).
- Create a Python project with an automated unit test.
- Use Git and GitHub for version control.
- Create a GitHub Actions workflow using YAML.
- Automatically run tests after a push.
- Demonstrate CI success, failure, and recovery.

2. CI Workflow

CI automatically validates code changes after they are integrated into a shared repository.

```
Developer → Code → Test → git commit → git push
                    ↓
                 GitHub
                    ↓
            GitHub Actions
                    ↓
      Checkout → Python → pytest
                    ↓
            PASS ✓ / FAIL ✗
```

3. Practical Steps

1. Create the project and Python virtual environment.
2. Install pytest.
3. Create hello.py and test_hello.py.
4. Run the test locally.
5. Initialize Git and commit the project.
6. Create the GitHub repository and push the code.
7. Create .github/workflows/ci.yml.
8. Push the workflow and open GitHub Actions.
9. Demonstrate a successful CI run.
10. Introduce an intentional test failure.
11. Push the failure and observe CI fail.
12. Fix the code, push again, and observe CI pass.

4. Create the Project

```
mkdir -p ~/kubeflow-mlops/ci-demo
cd ~/kubeflow-mlops/ci-demo
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install pytest
```

5. Application Code – hello.py

```
def greet(name):
    return f"Hello, {name}!"
```

```
if __name__ == "__main__":
    print(greet("MLOps Students"))
```

6. Test Code – test_hello.py

```
from hello import greet

def test_greet():
    assert greet("MLOps Students") == "Hello, MLOps Students!"
```

7. .gitignore

```
.venv/
__pycache__/
.pytest_cache/
```

8. Run Tests Locally

```
python -m pytest
```

Expected: 1 passed.

9. Initialize Git

```
git init
git branch -M main
git status
git add .
git commit -m "Add initial CI demo"
```

10. Create GitHub Repository

Create a repository named ci-demo on GitHub, then connect and push:

```
git remote add origin https://github.com/<your-username>/ci-demo.git
git push -u origin main
```

Security: never share your GitHub Personal Access Token.

11. Create the GitHub Actions Workflow

```
mkdir -p .github/workflows
nano .github/workflows/ci.yml
```

```
name: Python CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
```

```
steps:
  - name: Checkout code
    uses: actions/checkout@v4

  - name: Set up Python
    uses: actions/setup-python@v5
    with:
      python-version: "3.11"

  - name: Install dependencies
    run: |
      python -m pip install --upgrade pip
      pip install pytest

  - name: Run tests
    run: |
      pytest
```

12. Meaning of the Workflow

Element	Purpose
push / pull_request	Events that trigger CI
runs-on	Machine used to execute the job
checkout	Downloads repository code
setup-python	Configures Python
pip install pytest	Installs the test dependency
pytest	Runs automated tests

13. Push the CI Workflow

```
git add .github/workflows/ci.yml
git commit -m "Add GitHub Actions CI workflow"
git push
```

If a fine-grained PAT is used, repository Workflows permission may need Read and write access.

14. View CI on GitHub

13. Open the repository.
14. Select Actions.
15. Select Python CI.
16. Open the latest run.
17. Open the test job.

```
✓ Checkout code
✓ Set up Python
✓ Install dependencies
✓ Run tests
🟢 CI PASSED
```

15. Demonstrate CI Failure

Temporarily change hello.py from:

```
return f"Hello, {name}!"
```

to:

```
return f"Hi, {name}!"
```

Run:

```
python -m pytest
```

Expected: the test fails because it still expects the original greeting.

```
AssertionError
1 failed
```

```
git add hello.py
git commit -m "Introduce test failure for CI demo"
git push
```

GitHub Actions should show a failed run: 🔴 CI FAILED.

16. Fix and Re-run CI

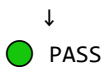
```
return f"Hello, {name}!"
```

```
python -m pytest
git add hello.py
git commit -m "Fix greeting test failure"
git push
```

GitHub Actions runs again and should show: 🟢 CI PASSED.

17. Complete CI Flow





18. Key CI Concepts

- Continuous Integration: frequently integrate and automatically validate code changes.
- Trigger: event that starts a workflow, such as push or pull_request.
- Runner: machine/environment where the workflow executes.
- Job: a collection of related workflow steps.
- Step: an individual action or command.
- Quality gate: a condition that must be satisfied before continuing.

19. CI in MLOps

```
Code Tests
   ↓
Data Validation
   ↓
Schema / Feature Validation
   ↓
Pipeline Tests
   ↓
Model Validation
   ↓
Quality Gate
   ↓
Continue / Stop
```

Example: if model accuracy is below the required threshold, the workflow can stop before delivery/deployment.

20. Quick Command Reference

```
# Environment
python3 -m venv .venv
source .venv/bin/activate
python -m pip install pytest

# Test
python -m pytest

# Git
git init
git branch -M main
git status
git add .
git commit -m "message"
git push

# Remote
git remote -v
```

21. Student Exercises

- Add a second test function and verify both tests execute.
- Change the greeting and update the test accordingly.
- Introduce an intentional error and observe CI fail.
- Fix the error and verify CI pass.
- Explain the purpose of each step in ci.yml.
- Draw the CI workflow from git push to PASS/FAIL.

22. Viva / Review Questions

18. What is Continuous Integration?
19. What triggers this workflow?
20. What is a GitHub Actions runner?
21. Why is actions/checkout used?
22. What does pytest do?
23. What happens when a CI test fails?
24. How do you fix a failed CI workflow?
25. How can CI be extended for an MLOps project?

23. Practical Outcome

After this practical, students should be able to create a simple Python CI pipeline, connect a local Git repository to GitHub, configure GitHub Actions, run automated tests, interpret successful and failed workflow runs, and explain the role of CI in an MLOps lifecycle.